

Unit II

Contents

1. Introduction to C language
2. History of C language
3. Simple C program
4. Identifiers
5. Basic Data types and sizes
6. Variables
7. Constants
8. Input and Output Statements
9. Operators
10. Expressions
11. Expression Evaluation
12. Precedence and Associativity
13. Type Conversions
14. Bitwise Operators
15. Selection statements
16. Repetition Statements
17. Examples
18. Exercises
19. Important Questions

INTRODUCTION TO C LANGUAGE

C is a structured programming language developed at AT & T's Bell Laboratories of USA in 1972. It was designed and written by Dennis Ritchie. In the late seventies C began to replace the more familiar languages of that time like PL/I, ALGOL, etc. C was originally designed for and implemented on the UNIX operating system on the DEC PDP-11, by Dennis Ritchie. The operating system, the C compiler, and essentially all UNIX applications are written in C. C is not tied to any particular hardware or system, however, and it is easy to write programs that will run without change on any machine that supports C.

HISTORY OF C LANGUAGE

Ken Thompson created a language which was based upon a language known as BCPL and it was called as B. B language was created in 1970, basically for UNIX operating system, Dennis Ritchie used ALGOL, BCPL and B as the basic reference language from which he created C. In 1983, the American National Standards Institute (ANSI) began the definition of a standard of a c. It was approved in December 1989. In 1990, The International standards Organization (ISO) adopted the ANSI standard. This version of c is known as C89.

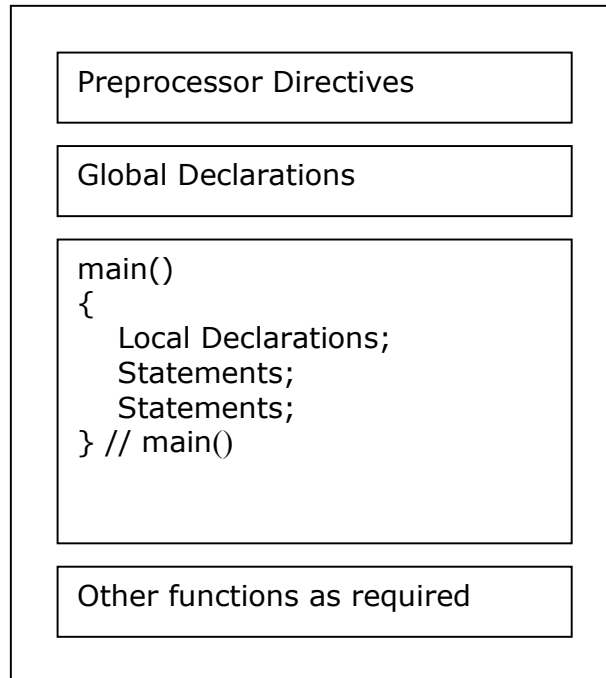
STRUCTURE OF A C PROGRAM

Every C program is made of one or more preprocessor commands, a global declaration section and one or more functions.

1. The preprocessor commands are special instructions to the preprocessor that specifies how to prepare the program for compilation. One of the most important preprocessor commands we use commonly is include. The include command tells the preprocessor that we need information from selected libraries known as header files.
2. The global declaration section comes at the beginning of the program. The global variables are declared here.
3. The execution of the program starts from the function called main(). All functions in a program, including main, are divided into two sections one declaration section and the statement section.
 - a) Declaration section: it is at the beginning of the function. It describes the data that you will be using in the function.

- b) Statement Section: It contains the instructions to the computer that perform specific task.

Structure of a C program



First C Program

```
#include <stdio.h>
void main()
{
    printf(" Hello! Welcome to C");
}
```

Comments: used to make our code unreadable means the compiler ignores these comments when it translates the program into executable code. C uses 2 different formats

1. Line commenting: it uses two slashes (//) to identify the comment and it is used to comment entire line.
2. Block Commenting: it uses two pair of tokens /* and */ to comment entire block of code.

C Character Set

The character set in C Language can be grouped into the following categories.

1. Letters
2. Digits
3. Special Characters
4. White Spaces

C Character-Set Table

Letters	Digits
Upper Case A to Z	0 to 9
Lower Case a to z	

Special Characters

,	Comma	&	Ampersand
.	Period	^	Caret
;	Semicolon	*	Asterisk
:	Colon	-	Minus Sign
?	Question Mark	+	Plus Sign
'	Aphostrophe	<	Opening Angle (Less than sign)
"	Quotation Marks	>	Closing Angle (Greater than sign)
!	Exclamation Mark	(	Left Parenthesis
	Vertical Bar	)	Right Parenthesis
/	Slash	[	Left Bracket
\	Backslash	]	Right Bracket
~	Tilde	{	Left Brace
_	Underscore	}	Right Brace
\$	Dollar Sign	#	Number Sign
%	Percentage Sign		

White Spaces are ignored by the compiler until they are a part of string constant. White Space may be used to separate words, but are strictly prohibited while using between characters of keywords or identifiers.

White Space

1. Blank Space
2. Horizontal Tab
3. Carriage Return
4. New Line
5. Form Feed

Keywords

Every word in C language is a keyword or an identifier. Keywords in C language cannot be used as a variable name. They are specifically used by the compiler for its own purpose and they serve as building blocks of a c program.

The following are the Keyword set of C language.

.auto	.else	.register	.union
.break	.enum	.return	.unsigned
.case	.extern	.short	.void
.char	.float	.signed	.volatile
.const	.for	.size of	.while
.continue	.goto	.static	.
.default	.if	.struct	.
.do	.int	.switch	.
.double	.long	.typedef	.

Identifiers

Identifiers refers to the name of user-defined variables, array and functions. A variable should be essentially a sequence of letters and or digits and the variable name should begin with a character.

Both uppercase and lowercase letters are permitted. The underscore character is also permitted in identifiers.

The identifiers must conform to the following rules.

1. First character must be an alphabet (or underscore)
2. Identifier names must consists of only letters, digits and underscore.
3. A identifier name should have less than 40 characters.
4. Any standard C language keyword cannot be used as a variable name.
5. A identifier should not contain a space.

Basic Data Types

A type defines a set of values and set of operations that can be applied on those values.

C language data types can be broadly classified as

Primary data type

Derived data type

User-defined data type

Primary data type:

All C Compilers accept the following fundamental data types

1.	Integer	int
2.	Character	char
3.	Floating Point	float
4.	Double precision floating point	double
5.	Void	void

The size and range of each data type is given in the table below

DATA TYPE	RANGE OF VALUES
char	-128 to 127
int	-32768 to +32767
float	3.4 e-38 to 3.4 e+38
double	1.7 e-308 to 1.7 e+308

Integer Type:

Integers are whole numbers with a machine dependent range of values. C has 3 classes of integer storage namely short int, int and long int. All of these data types have signed and unsigned forms. A short int requires half the space than normal integer values. Unsigned numbers are always positive and consume all the bits for the magnitude of the number. The long and unsigned integers are used to declare a longer range of values.

Floating Point Types:

Floating point number represents a real number with 6 digits precision. Floating point numbers are denoted by the keyword float. When the accuracy of the floating point number is insufficient, we can use the double to define the number. The double is same as float but with longer precision. To extend the precision further we can use long double which consumes 80 bits of memory space.

Character Type:

A single character can be defined as a defined as a character type of data. Characters are usually stored in 8 bits of internal storage. The qualifier signed or unsigned can be explicitly applied to char. While unsigned characters have values between 0 and 255, signed characters have values from -128 to 127.

Size and Range of Data Types on 16 bit machine.

type	SIZE (Bits)	Range
char or signed char	8	-128 to 127
unsigned char	8	0 to 255
int or signed int	16	-32768 to 32767
unsigned int	16	0 to 65535
short int or signed short int	8	-128 to 127
unsigned short int	8	0 to 255
long int or signed long int	32	-2147483648 to 2147483647
unsigned long int	32	0 to 4294967295
float	32	3.4 e-38 to 3.4 e+38
double	64	1.7e-308 to 1.7e+308
long double	80	3.4 e-4932 to 3.4 e+4932

Variables:

Variable is an identifier used to store the value of a type in the memory.

Variable Declaration: Each variable in your program must be declared and defined.

The declaration does two things.

1. Tells the compiler the variables name.
2. Specifies what type of data the variable will hold.

The general format of any declaration:

datatype v1, v2, v3, vn;

Where v1, v2, v3 are variable names. Variables are separated by commas. A declaration statement must end with a semicolon.

Example:

int sum;

int number, salary;

double average, mean;

Datatype	Keyword Equivalent
Character	char
Unsigned Character	unsigned char
Signed Character	signed char
Signed Integer	signed int (or) int
Signed Short Integer	signed short int (or) short int (or) short
Signed Long Integer	signed long int (or) long int (or) long
UnSigned Integer	unsigned int (or) unsigned
UnSigned Short Integer	unsigned short int (or) unsigned short
UnSigned Long Integer	unsigned long int (or) unsigned long
Floating Point	float
Double Precision Floating Point	double
Extended Double Precision Floating Point	long double

Constants:

A constant value is the one which does not change during the execution of a program. C supports several types of constants.

1. Integer Constants
2. Real Constants
3. Single Character Constants
4. String Constants

Integer Constants:

An integer constant is a sequence of digits. There are 3 types of integers namely decimal integer, octal integers and hexadecimal integer.

Decimal Integers consists of a set of digits 0 to 9 preceded by an optional + or - sign. Spaces, commas and non digit characters are not permitted between digits. Example for valid decimal integer constants are

123

-31

0

562321

+ 78

Octal Integers constant consists of any combination of digits from 0 through 7 with a 0 at the beginning. Some examples of octal integers are

026

0

0347

0676

Hexadecimal integer constant is preceded by 0X or 0x, they may contain alphabets from A to F or a to f. The alphabets A to F refers to 10 to 15 in decimal digits. Example of valid hexadecimal integers are

0X2

0X8C

0Xbcd

0x

Real Constants:

Real Constants consists of a fractional part in their representation. Integer constants are inadequate to represent quantities that vary continuously. These quantities are represented by numbers containing fractional parts like 26.082. Example of real constants are

0.0026

-0.97

435.29

+487.0

Single Character Constants:

A Single Character constant represent a single character which is enclosed in a pair of quotation symbols.

Example for character constants are '5' 'x' ';' '`

All character constants have an equivalent integer value which is called ASCII Values.

String Constants:

A string constant is a set of characters enclosed in double quotation marks. The characters in a string constant sequence may be a alphabet, number, special character and blank space. Example of string constants are

"VISHAL"

"1234"

"God Bless"

"!.....?"

Backslash Character Constants [Escape Sequences]:

Backslash character constants are special characters used in output functions. Although they contain two characters they represent only one character. Given below is the table of escape sequence and their meanings.

Constant	Meaning
'\a'	.Audible Alert (Bell)
'\b'	.Backspace
'\f'	.Formfeed
'\n'	.New Line
'\r'	.Carriage Return
'\t'	.Horizontal tab
'\v'	.Vertical Tab
'\''	.Single Quote
'\"'	.Double Quote
'\?'	.Question Mark
'\\'	.Back Slash
'\0'	.Null

C Programming - Managing Input and Output Operations

One of the essential operations performed in a C language programs is to provide input values from keyboard to the program and output the data produced by the program to a standard output device. We can assign values to variable through assignment statements such as `x = 5` `a = 0` ; and so on.

Another method to Input data the function we use `scanf()` function which can be used to read data from a key board. For outputting results we have used extensively the function `printf()` which sends results out to a terminal. There exists several functions in 'C' language that can carry out input output operations. These functions are collectively known as standard Input/Output Library. Each program that uses standard input / output function must contain the statement.

include < stdio.h > at the beginning.

Single character input output:

The basic operation done in input output is to read a character from the standard input device such as the keyboard and to output or writing it to the output unit usually the screen. The `getchar()` function can be used to read a character from the standard input device. The `scanf()` can also be used to achieve the function.

The `getchar()` has the following form.

`variable name = getchar();`

Variable name is a valid 'C' variable, that has been declared already and that possess the type char.

The `putchar` function which is analogous to `getchar` function can be used for writing characters one at a time to the output terminal.

The general form is:

`putchar (variable name);`

Where variable is a valid C type variable that has already been declared Ex:-

`putchar(name);`

Displays the value stored in variable C to the standard screen.

Formatted Input For scanf():

The formatted input refers to input data that has been arranged in a particular format. Input values are generally taken by using the `scanf` function.

The `scanf` function has the general form.

`scanf ("Control string", arg1, arg2, arg3 .....argn);`

The format field is specified by the control string and the arguments

`arg1, arg2, .....argn` specifies the address of location where address is to be stored.

The control string specifies the field format which includes format specifications and optional number specifying field width and the conversion character % and also blanks, tabs and newlines.

The Blanks tabs and newlines are ignored by compiler. The conversion character % is followed by the type of data that is to be assigned to variable of the assignment. The field width specifier is optional.

The general format for reading a integer number is: % x d

Here percent sign (%) denotes that a specifier for conversion follows and x is an integer number which specifies the width of the field of the number that is being

read. The data type character d indicates that the number should be read in integer mode.

Example :

```
scanf ("%3d %4d", &sum1, &sum2);
```

If the values input are 175 and 1342 here value 175 is assigned to sum1 and 1342 to sum 2. Suppose the input data was follows 1342 and 175.

The number 134 will be assigned to sum1 and sum2 has the value 2 because of %3d the number 1342 will be cut to 134 and the remaining part is assigned to second variable sum2. If floating point numbers are assigned then the decimal or fractional part is skipped by the computer.

To read the long integer data type we can use conversion specifier % ld & % hd for short integer.

Input specifications for real number:

Field specifications are not to be use while representing a real number therefore real numbers are specified in a straight forward manner using % f specifier.

The general format of specifying a real number input is

```
Scanf (% f ", &variable);
```

Example:

```
Scanf ("%f %f % f", &a, &b, &c);
```

With the input data

321.76, 4.321, 678 The values

321.76 is assigned to a , 4.321 to b & 678 to C.

If the number input is a double data type then the format specifier should be % lf instead of %f.

Input specifications for a character.

Single character or strings can be input by using the character specifiers. The general format is % xc or %xs

Where C and S represents character and string respectively and x represents the field width.

The address operator need not be specified while we input strings.

Example :

```
Scanf ("%C %15C", &ch, nname):
```

Here suppose the input given is a, Robert then a is assigned to ch and name will be assigned to Robert.

Printing One Line:

```
printf();
```

The most simple output statement can be produced in C' Language by using printf statement. It allows you to display information required to the user and also prints the variables we can also format the output and provide text labels. The simple statement such as

```
Printf ("Enter 2 numbers");
```

Prompts the message enclosed in the quotation to be displayed.

Conversion Strings and Specifiers:

The printf () function is quite flexible. It allows a variable number of arguments, labels and sophisticated formatting of output. The general form of the printf () function is

Syntax

```
printf ("conversion string", variable list);
```

The conversion string includes all the text labels, escape character and conversion specifiers required for the desired output. The variable includes all the variable to be printed in order they are to be printed. There must be a conversion specifies after each variable.

Specifier Meaning

%c – Print a character

%d – Print a Integer

%i – Print a Integer

%e – Print float value in exponential form.

%f – Print float value

%g – Print using %e or %f whichever is smaller

%o – Print actual value

%s – Print a string

%x – Print a hexadecimal integer (Unsigned) using lower case a – F

%X – Print a hexadecimal integer (Unsigned) using upper case A – F

%a – Print a unsigned integer.

%p – Print a pointer value

%hx – hex short

%lo – octal long

%ld – long

OPERATORS

An operator is a symbol which helps the user to give instruction to the computer to do a certain mathematical or logical manipulations. Operators are used in C language program to operate on data and variables.

C has a rich set of operators which can be classified as follows:

1. Arithmetic operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Increments and Decrement Operators
6. Conditional Operators
7. Bitwise Operators
8. Special Operators

1. Arithmetic Operators

All the basic arithmetic operations can be carried out in C. All the operators have almost the same meaning as in other languages. Both unary and binary operations are available in C language. Unary operations operate on a single operand, therefore the number 5 when operated by unary – will have the value –5.

Arithmetic Operators

Operator	Meaning
+	Addition or Unary Plus
–	Subtraction or Unary Minus
*	Multiplication
/	Division
%	Modulus Operator

Examples of arithmetic operators are:

$x + y$, $x - y$, $-x + y$, $a * b + c$, $-a * b$ etc.,

here a , b , c , x , y are known as operands. The modulus operator is a special operator in C language which evaluates the remainder of the operands after division.

Integer Arithmetic

When an arithmetic operation is performed on two whole numbers or integers than such an operation is called as integer arithmetic. It always gives an integer as the result. Let $x = 27$ and $y = 5$ be 2 integer numbers. Then the integer operation leads to the following results.

$x + y = 32$, $x - y = 22$, $x * y = 115$, $x \% y = 2$, $x / y = 5$

In integer division the fractional part is truncated.

Floating point arithmetic

When an arithmetic operation is performed on two real numbers or fraction numbers such an operation is called floating point arithmetic. The floating point results can be truncated according to the properties requirement. The remainder operator is not applicable for floating point arithmetic operands.

Let $x = 14.0$ and $y = 4.0$ then

$x + y = 18.0$, $x - y = 10.0$, $x * y = 56.0$, $x / y = 3.50$

Mixed mode arithmetic

When one of the operand is real and other is an integer and if the arithmetic operation is carried out on these 2 operands then it is called as mixed mode arithmetic. If any one operand is of real type then the result will always be real thus $15/10.0 = 1.5$

2. Relational Operators

Often it is required to compare the relationship between operands and bring out a decision and program accordingly. This is when the relational operator come into picture. C supports the following relational operators.

Operator	Meaning
<	is less than
<=	is less than or equal to
>	is greater than
>=	is greater than or equal to
==	is equal to
!=	is not equal to

It is required to compare the marks of 2 students, salary of 2 persons, we can compare them using relational operators.

A simple relational expression contains only one relational operator and takes the following form.

exp1 relational operator exp2

Where exp1 and exp2 are expressions, which may be simple constants, variables or combination of them. Given below is a list of examples of relational expressions and evaluated values.

6.5 <= 25 TRUE , -65 > 0 FALSE , 10 < 7 + 5 TRUE

Relational expressions are used in decision making statements of C language such as if, while and for statements to decide the course of action of a running program.

3. Logical Operators

C has the following logical operators; they compare or evaluate logical and relational expressions.

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

Logical AND (&&)

This operator is used to evaluate 2 conditions or expressions with relational operators simultaneously. If both the expressions to the left and to the right of the logical operator is true then the whole compound expression is true.

Example: `a > b && x == 10`

The expression to the left is `a > b` and that on the right is `x == 10` the whole expression is true only if both expressions are true i.e., if `a` is greater than `b` and `x` is equal to 10.

Logical OR (||)

The logical OR is used to combine 2 expressions or the condition evaluates to true if any one of the 2 expressions is true.

Example: `a < m || a < n`

The expression evaluates to true if any one of them is true or if both of them are true. It evaluates to true if `a` is less than either `m` or `n` and when `a` is less than both `m` and `n`.

Logical NOT (!)

The logical not operator takes single expression and evaluates to true if the expression is false and evaluates to false if the expression is true. In other words it just reverses the value of the expression.

For example

`!(x >= y)` the NOT expression evaluates to true only if the value of `x` is neither greater than or equal to `y`

4. Assignment Operators

The Assignment Operator evaluates an expression on the right of the expression and substitutes it to the value or variable on the left of the expression.

Example: $x = a + b$

Here the value of $a + b$ is evaluated and substituted to the variable x . In addition, C has a set of shorthand assignment operators of the form.

$\text{var oper} = \text{exp};$

Here var is a variable, exp is an expression and oper is a C binary arithmetic operator. The operator $\text{oper} =$ is known as shorthand assignment operator

Example: $x += 1$ is same as $x = x + 1$

The commonly used shorthand assignment operators are as follows

Shorthand assignment operators

Statement with simple assignment operator	Statement with shorthand operator
$a = a + 1$	$a += 1$
$a = a - 1$	$a -= 1$
$a = a * (n+1)$	$a *= (n+1)$
$a = a / (n+1)$	$a /= (n+1)$
$a = a \% b$	$a \% = b$

5. Increment and Decrement Operators

The increment and decrement operators are one of the unary operators which are very useful in C language. They are extensively used in for and while loops. The syntax of the operators is given below

1. $++ \text{ variable name}$
2. $\text{variable name}++$
3. $-- \text{ variable name}$
4. $\text{variable name}--$

The increment operator $++$ adds the value 1 to the current value of operand and the decrement operator $--$ subtracts the value 1 from the current value of

operand. ++variable name and variable name++ mean the same thing when they form statements independently, they behave differently when they are used in expression on the right hand side of an assignment statement.

Consider the following

```
m = 5;
```

```
y = ++m; (prefix)
```

In this case the value of y and m would be 6

Suppose if we rewrite the above statement as

```
m = 5;
```

```
y = m++; (post fix)
```

Then the value of y will be 5 and that of m will be 6. A prefix operator first adds 1 to the operand and then the result is assigned to the variable on the left. On the other hand, a postfix operator first assigns the value to the variable on the left and then increments the operand.

6. Conditional or Ternary Operator

The conditional operator consists of 2 symbols the question mark (?) and the colon (:). The syntax for a ternary operator is as follows:

```
exp1 ? exp2 : exp3
```

The ternary operator works as exp1 is evaluated first. If the expression is true then exp2 is evaluated & its value becomes the value of the expression. If exp1 is false, exp3 is evaluated and its value becomes the value of the expression. Note that only one of the expression is evaluated.

For example:

```
a = 10;
```

```
b = 15;
```

```
x = (a > b) ? a : b
```

Here x will be assigned to the value of b. The condition follows that the expression is false therefore b is assigned to x.

Output

Input 2 integers : 34 45

The largest of two numbers is 45

7. Bitwise Operators

C has a distinction of supporting special operators known as bitwise operators for manipulation data at bit level. A bitwise operator operates on each bit of data. Those operators are used for testing, complementing or shifting bits to the right or left. Bitwise operators may not be applied to a float or double.

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise Exclusive
<<	Shift left
>>	Shift right

8. Special Operators

C supports some special operators of interest such as comma operator, size of operator, pointer operators (& and *) and member selection operators (. and ->). The size of and the comma operators are discussed here.

The Comma Operator

The comma operator can be used to link related expressions together. A comma-linked list of expressions are evaluated left to right and value of right most expression is the value of the combined expression.

For example the statement

```
value = (x = 10, y = 5, x + y);
```

First assigns 10 to x and 5 to y and finally assigns 15 to value. Since comma has the lowest precedence in operators the parenthesis is necessary. Some examples of comma operator are

In for loops:

```
for (n=1, m=10, n <=m; n++,m++)
```

In while loops

```
While (c=getchar(), c != '10')
```

Exchanging values

```
t = x, x = y, y = t;
```

The size of Operator

The operator size of gives the size of the data type or variable in terms of bytes occupied in the memory. The operand may be a variable, a constant or a data type qualifier.

Example

```
m = sizeof (sum);
```

```
n = sizeof (long int);
```

```
k = sizeof (235L);
```

The size of operator is normally used to determine the lengths of arrays and structures when their sizes are not known to the programmer. It is also used to allocate memory space dynamically to variables during the execution of the program.

EXPRESSIONS

An expression is a combination of variables constants and operators written according to the syntax of C language. In C every expression evaluates to a value i.e., every expression results in some value of a certain type that can be assigned to a variable. Some examples of C expressions are shown in the table given below.

Algebraic Expression	C Expression
$a \times b - c$	$a * b - c$
$(m + n) (x + y)$	$(m + n) * (x + y)$
(ab / c)	$a * b / c$
$3x^2 + 2x + 1$	$3*x*x+2*x+1$
$(x / y) + c$	$x / y + c$

Rules for evaluation of expression

1. First parenthesized sub expression left to right are evaluated.
2. If parenthesis are nested, the evaluation begins with the innermost sub expression.
3. The precedence rule is applied in determining the order of application of operators in evaluating sub expressions.
4. The associability rule is applied when two or more operators of the same precedence level appear in the sub expression.
5. Arithmetic expressions are evaluated from left to right using the rules of precedence.
6. When Parenthesis are used, the expressions within parenthesis assume highest priority.

Evaluation of Expressions

Expressions are evaluated using an assignment statement of the form

Variable = expression;

Variable is any valid C variable name. When the statement is encountered, the expression is evaluated first and then replaces the previous value of the variable on the left hand side. All variables used in the expression must be assigned values before evaluation is attempted.

Example of evaluation statements are

$x = a * b - c$, $y = b / c * a$, $z = a - b / c + d$;

The following program illustrates the effect of presence of parenthesis in expressions.

Precedence in Arithmetic Operators

An arithmetic expression without parenthesis will be evaluated from left to right using the rules of precedence of operators. There are two distinct priority levels of arithmetic operators in C.

High priority * / %

Low priority + -

OPERATOR PRECEDENCE AND ASSOCIATIVITY

Each operator in C has a precedence associated with it. The precedence is used to determine how an expression involving more than one operator is evaluated. There are distinct levels of precedence and an operator may belong to one of these levels. The operators of higher precedence are evaluated first.

The operators of same precedence are evaluated from right to left or from left to right depending on the level. This is known as Associativity property of an operator.

The table given below gives the precedence of each operator.

Order	Category	Operator	Operation	Associativity
1	Highest precedence	() [] → :: .	Function call	L → R Left to Right
2	Unary	! ~ + - ++ -- & * Size of	Logical negation (NOT) Bitwise 1's complement Unary plus Unary minus Pre or post increment Pre or post decrement Address Indirection Size of operand in bytes	R → L Right -> Left
3	Member Access	.* →*	Dereference Dereference	L → R
4	Multiplication	* / %	Multiply Divide Modulus	L → R
5	Additive	+ -	Binary Plus Binary Minus	L → R
6	Shift	<< >>	Shift Left Shift Right	L → R
7	Relational	< <= > >=	Less than Less than or equal to Greater than Greater than or equal to	L → R
8	Equality	== !=	Equal to Not Equal to	L → R
9	Bitwise AAND	&	Bitwise AND	L → R
10	Bitwise XOR	^	Bitwise XOR	L → R
11	Bitwise OR		Bitwise OR	L → R
12	Logical AND	&&	Logical AND	L → R
13	Conditional	? :	Ternary Operator	R → L

14	Assignment	= *= %= /=	Assignment Assign product Assign reminder Assign quotient	R → L
		+= -= &= ^= =	Assign sum Assign difference Assign bitwise AND Assign bitwise XOR Assign bitwise OR	
		<<= >>=	Assign left shift Assign right shift	
15	Comma	,	Evaluate	L → R

TYPE CONVERSIONS IN EXPRESSIONS

Implicit type conversion

C permits mixing of constants and variables of different types in an expression. C automatically converts any intermediate values to the proper type so that the expression can be evaluated without losing any significance. This automatic type conversion is known as implicit type conversion.

During evaluation it adheres to very strict rules and type conversion. If the operands are of different types the lower type is automatically converted to the higher type before the operation proceeds. The result is of higher type. The following rules apply during evaluating expressions.

All short and char are automatically converted to int then

1. If one operand is long double, the other will be converted to long double and resultwill be long double.
2. If one operand is double, the other will be converted to double and result will be double.
3. If one operand is float, the other will be converted to float and result will be float.
4. If one of the operand is unsigned long int, the other will be converted into unsignedlong int and result will be unsigned long int.
5. If one operand is long int and other is unsigned int thena. If unsigned int can be converted to long int, then unsigned int operand will be.....converted as such and the result will be long int.....b. Else Both operands will be converted to unsigned long int and the result will beunsigned long int.

6. If one of the operand is long int, the other will be converted to long int and the result will be long int.
7. If one operand is unsigned int the other will be converted to unsigned int and theresult will be unsigned int.

Explicit Conversion

Many times there may arise a situation where we want to force a type conversion in a way that is different from automatic conversion. The process of such a local conversion is known as explicit conversion or casting a value. The general form is

variable= (type_name) expression

Bitwise operators

C has two categories of bitwise operators that operate on data at the bit level: Logical bitwise operators and shift bitwise operators.

Logical Bitwise Operators

Logical bitwise operators look data as individual bits to be manipulated. There are four operators first three bitwise and, bitwise inclusive or and bitwise exclusive or are binary operators and last one is one's complement is a unary operator.

First Operand bit	Second operand bit	Result
0	0	0
0	1	0
1	0	0
1	1	1

Truth table of Logical Bitwise And

First Operand bit	Second operand bit	Result
0	0	0
0	1	1
1	0	1
1	1	1

Truth table of Logical Bitwise Inclusive Or

First Operand bit	Second operand bit	Result
0	0	0
0	1	1
1	0	1
1	1	0

Truth table of Logical Bitwise Exclusive Or

Original bit	Result
0	1
1	0

Truth table of One's Complement Operator

Shift Operators

The shift operators move bits to the right or left. When applied to unsigned numbers, these operators are implementation independent. Therefore they must be used with caution with signed numbers.

Bitwise Left Shift Operator (<<)

It is a binary operator that requires two integral operands. The first operand is the value to be shifted. The second operand specifies the number of bits to be shifted. Shifting binary numbers are like shifting decimal numbers. If we have an eight digit decimal number, and we shift it three places to the left, then the leftmost three digits are lost and three zero digits are added to the right.

Bitwise Right Shift Operator (>>)

It is a binary operator that requires two integral operands. The first operand is the value to be shifted. The second operand specifies the number of bits to be shifted. Shifting binary numbers are like shifting decimal numbers. When bits are shifted right, the bits at right most end are deleted.

STATEMENTS IN C

A statement causes an action to be performed by the program. It translates directly into one or more executable computer instructions.

C defines several types of statements.

Types of Statements

Statements in C are categorized into following types-

1. Selection/Conditional Statement - They decide the flow of statements on based on evaluation of results of conditions. if - else and switch statements come under this category.

C uses the keyword **if** to implement the decision control instruction. The general form of **if** statement looks like this:

**a) if (this condition is true)
 execute this statement ;**

The keyword **if** tells the compiler that what follows is a decision control instruction. The condition following the keyword **if** is always enclosed within a pair of parentheses. If the condition, whatever it is, is true, then the statement is executed. If the condition is not true then the statement is not executed;

```
/* Demonstration of if statement */
main( )
{
    int num ;
    printf ( "Enter a number less than 10 " ) ;
    scanf ( "%d", &num ) ;

    if ( num <= 10 )
        printf ( "Thanks! For Enter value < 10!" ) ;
}
```

```
/* Demonstration of if statement with multiple statements*/
main( )
{
    int num ;
    printf ( "Enter a number less than 10 " ) ;
    scanf ( "%d", &num ) ;

    if ( num <= 10 )
    {
        printf ( "Given number is %d",num);
        printf("\n Thanks! For Enter value < 10!" ) ;
    }
}
```

b) if (condition)
 do this ;
else
 do this ;

In the above if statement the condition compares first, if it is true then statement in true part will executes, if the specified condition is false then the condition in else part will executes.

```
/* Demonstration of if-else statement */
main( )
{
int num ;
printf ( "Enter a number " ) ;
scanf ( "%d", &num ) ;
if ( num % 2 == 0 )
printf ( "\n Given number is Even" ) ;
else
printf ( "\n Given number is Odd" ) ;
}
```

c) if (condition)
 {
 do this ;
 and this ;
 }
else
 {
 do this ;
 and this ;
 }

In the above if statement the condition compares first, if it is true then statement in true part block will executes, if the specified condition is false then the condition in else part block will executes.

```
/* Demonstration of if-else statement with multiple statements in each
block*/
main( )
{
int num1,num2 ;
printf ( "Enter any two numbers " ) ;
scanf ( "%d %d", &num1,&num2 ) ;
if ( num1 > num2 )
{
printf ( "\n %d number is Max" ,num1 ) ;
printf ( "\n %d number is Min",num2 ) ;
}
else
{
printf ( "\n %d number is Max" ,num2 ) ;
printf ( "\n %d number is Min",num1 ) ;
}
}
```

```

    }
d) if ( condition )
    do this ;
    else
    {
    if ( condition )
    do this ;
    else
    {
    do this ;
    and this ;
    }
    }

```

In the above type first the condition checks, if the condition is true then it executes the statement of true part, if the condition fails the it enters into the else block, in else there is another if statement, it also checks the condition in else block and if the condition true executes the true part or else follow the same process for the rest.

Note: above statement is also called as else – if ladder.

else – if ladder means if we occurs more conditions on else part of each if statement repeatedly is called else – if ladder

```

/* Demonstration of else – if ladder*/
main( )
{
float avg;
printf ( "Enter your average marks " ) ;
scanf ( "%f", &avg ) ;

if ( avg>=80 )
    printf ( "\nYou got Distinction");
    else if (avg>=70)
        printf ( "\nYou got high First class");
        else if (avg>=60)
            printf ( "\nYou got First class");
            else if (avg>=50)
                printf ( "\nYou got Second class");
                else if (avg>=35)
                    printf ( "\nYou got Third class");
}

```

```

e) if ( condition )
{
    if ( condition )
    do this ;
    else
    {
    do this ;
    and this ;
    }
}
else
do this ;

```

In the above type first the condition checks, if the condition is true then it enter into the sub condition and executes the statement of true part of sub condition, if the condition is true then the statements are executed, if the condition fails then it enters into the else block, and executes the statements in else part and follows the same process for the remaining statements till the main if statement closes.

Note: Above type of statement is also called as Nested if.

Nested if means if we specify an if statement in another if statement repeatedly.

```
/* Demonstration of Nested - if*/
main( )
{
int a,b,c;
printf ( "Enter any three integers" ) ;
scanf ( "%d %d %d ", &a, &b, &c ) ;

if (a == b)
    if( b==c)
        printf("\n All the numbers are same");
    else
        printf("\n All the numbers are different");
}
```

2. Iteration Statements - These are used to run a particular block statement repeatedly or in other words form a loop.

There are three methods by way of which we can repeat a part of program.

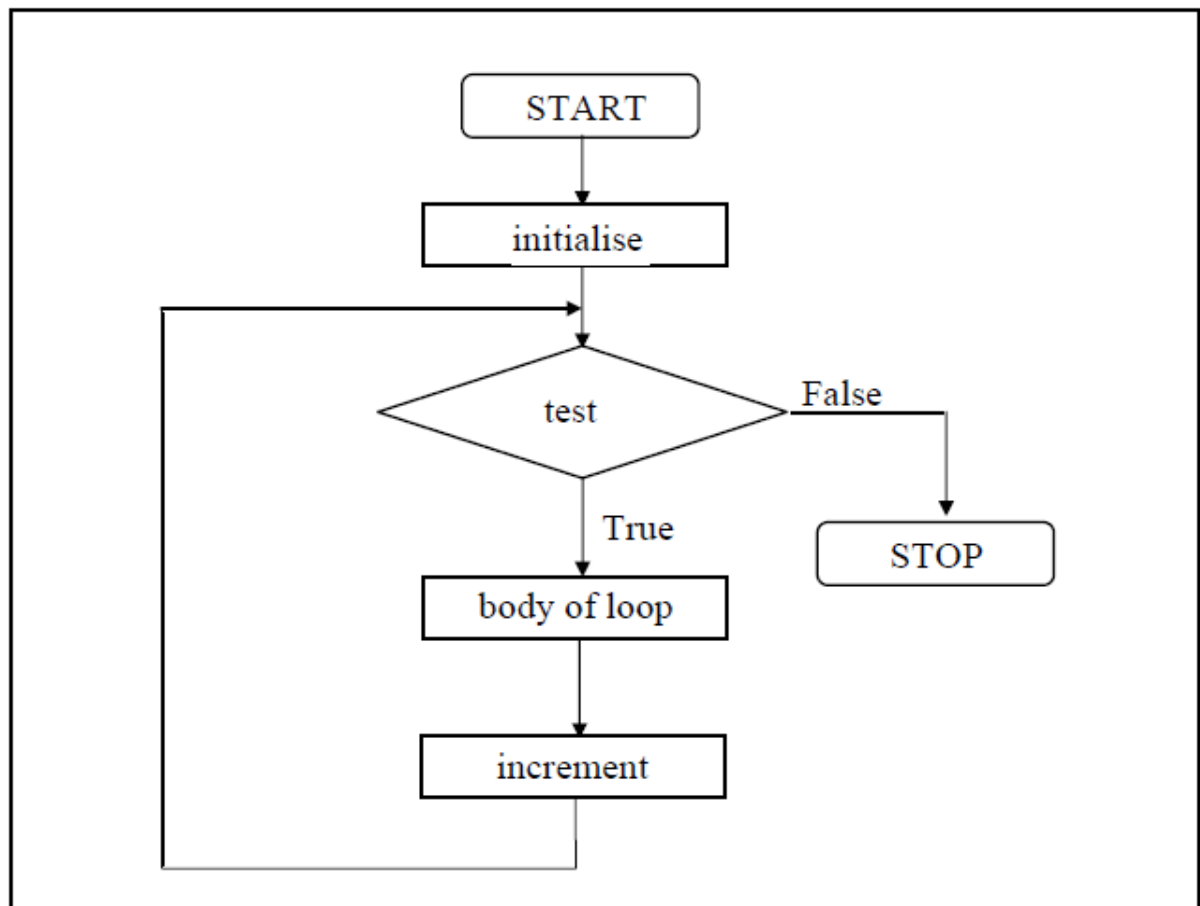
They are:

- (a) Using a while statement
- (b) Using a for statement
- (c) Using a do-while statement

- **While Statement:** It is an iteration statement which test the given condition in the beginning of the block, if the condition is true then the statements inside the block are executed, if the condition is false then the statements after the block are executed. This statement is called as an Entry Criteria Iteration Statement.

The general form of while is as shown below:

```
initialise loop counter ;
When the while ( test loop counter using a condition )
{
do this ;
and this ;
increment loop counter ;
}
```



/* Calculation of simple interest for 3 sets of p, n and r */

main()

{

int p, n, count ;

float r, si ;

count = 1 ;

while (count <= 3)

{

printf ("\nEnter values of p, n and r ") ;

scanf ("%d %d %f", &p, &n, &r) ;

si = p * n * r / 100 ;

count = count +1;

printf ("Simple interest = Rs. %f", si) ;

}

getch();

}

Output:

Enter values of p, n and r 1000 5 13.5

Simple interest = Rs. 675.000000

Enter values of p, n and r 2000 5 13.5

Simple interest = Rs. 1350.000000

Enter values of p, n and r 3500 5 3.5

Simple interest = Rs. 612.500000

- **For Statement:** It is an iteration statement which is similar to while statement but this statement contains three expressions first one is initializing the loop counter, second expression is the condition and third one is loop counter increment/decrement. This statement first initialize the loop counter and then test the given condition in the beginning of the block, if the condition is true then the statements inside the block are executed, and then the counter increment works. if the condition is false then the statements after the block are executed. This statement is called as an Entry Criteria Iteration Statement.

The general form of for statement is as shown below:

```
for ( initialize loop counter ; test loop counter using a condition ;  
increment loop counter )
```

```
{  
do this ;  
and this ;  
}
```

```
/* Calculation of simple interest for 3 sets of p, n and r */
```

```
main ( )
```

```
{
```

```
    int p, n, count ;
```

```
    float r, si ;
```

```
    for ( count = 1 ; count <= 3 ; count = count + 1 )
```

```
    {
```

```
        printf ( "Enter values of p, n, and r " ) ;
```

```
        scanf ( "%d %d %f", &p, &n, &r ) ;
```

```
        si = p * n * r / 100 ;
```

```
        printf ( "Simple Interest = Rs.%f\n", si ) ;
```

```
    }
```

```
}
```

Output:

Enter values of p, n and r 1000 5 13.5

Simple interest = Rs. 675.000000

Enter values of p, n and r 2000 5 13.5

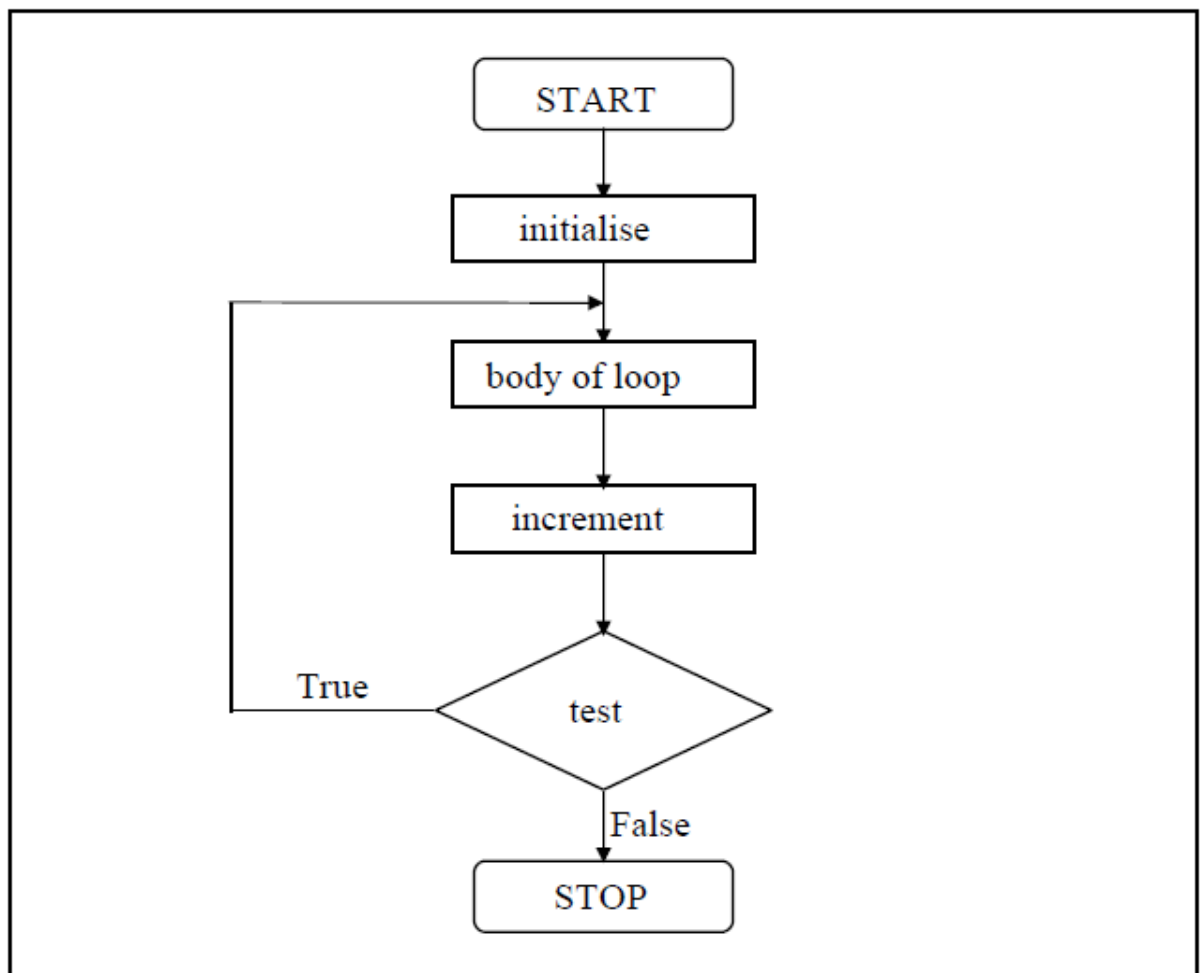
Simple interest = Rs. 1350.000000

Enter values of p, n and r 3500 5 3.5

- **Do While Statement:** This statement first initialize the loop counter, then the statements inside the block are executed for the first iteration and then test the given condition at the ending of the block, if the condition is true then the statements in the loop are executed, if the condition is false then the statements after the block are executed. This statement is called as an Exit Criteria Iteration Statement.

The general form of for statement is as shown below:

```
initialize loop counter;  
do  
{  
do this ;  
and this ;  
increment loop counter;  
} while (test loop counter using a condition);
```



```

/* demonstrate the do while statement */
void main( )
{
    char another ;
    int num ;
    do
    {
        printf ( "Enter a number " ) ;
        scanf ( "%d", &num ) ;
        printf ( "square of %d is %d", num, num * num ) ;
        printf ( "\nWant to enter another number y/n " ) ;
        scanf ( " %c", &another ) ;
    } while ( another == 'y' ) ;
}

```

3. Jump Statements - These are used to make the flow of your statements from one point to another. break, continue, goto and return come under this category.

The break Statement

If we want to jump out of a loop instantly, without waiting to get back to the conditional test. The keyword **break** allows us to do this. When **break** is encountered inside any loop, control automatically passes to the first statement after the loop. A **break** is usually associated with an **if**.

/* Program to find a given number is prime or not */

```

main( )
{
    int num, i ;
    printf ( "Enter a number " ) ;
    scanf ( "%d", &num ) ;
    i = 2 ;
    while ( i <= num - 1 )
    {
        if ( num % i == 0 )
        {
            printf ( "Not a prime number" ) ;
            break ;
        }
        i++ ;
    }
    printf ( "Prime number" ) ;
}

```

The continue Statement

In some programming situations we want to take the control to the beginning of the loop, bypassing the statements inside the loop, which have not yet been executed. The keyword **continue** allows us to do this. When **continue** is encountered inside any loop, control automatically passes to the beginning of the loop.

```
main( )
{
int i, j ;
for ( i = 1 ; i <= 2 ; i++ )
{
for ( j = 1 ; j <= 2 ; j++ )
{
if ( i == j )
continue ;
printf ( "\n%d %d\n", i, j ) ;
}
}
}
```

The output of the above program would be...

```
1 2
2 1
```

The goto Statement

In some programming situations we want to take the control to the desired location within the programming, bypassing the statements in a program which are no need to be executed. The keyword **goto** allows us to do this. When **goto** is encountered, control automatically passes to the label.

Backward Jump:

label:

statements;

statements;

goto label;

Forward Jump:

goto label;

statements;

statements;

label: